



OUT OF OFFICE HOURS

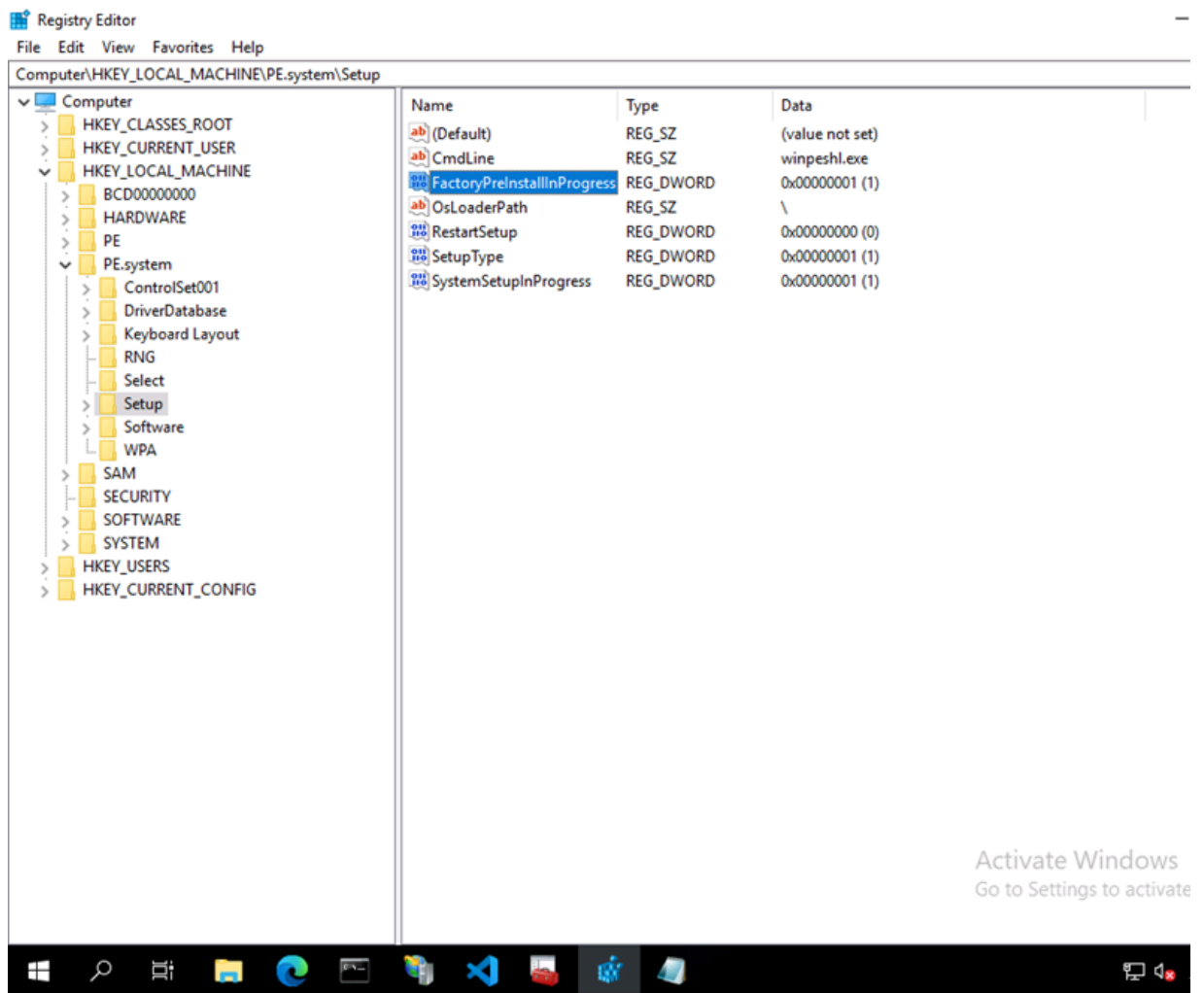
[Home](#)[RSS](#)[SUBSCRIBE](#)[Log in](#)

Windows PE startup, revisited

Windows 10

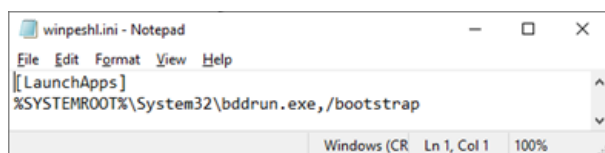
Way back when, I had to decipher the mechanism used in Windows PE to run stuff, i.e. to kick off the MDT/BDD processes. It's been years since I looked at that, because once you do it and it "just works" you don't worry about it much any more. But I had an opportunity recently to "re-learn" how that works. There is [some documentation that explains this](#), but given that it's Windows Vista-era and no longer being maintained, there's no guarantee that it will continue to exist. Plus, there's some room to expand on its description.

You probably wouldn't be surprised to know that Windows PE shares a lot of code with "full" Windows, so the startup process is fairly similar. Eventually, it gets to the point where the winlogon.exe process is started, and that (eventually) enables users to sign in and access the desktop. But before it does that, it will run whatever command it finds in HKLM\System\Setup\CmdLine. That happens at least when certain SetupType, FactoryPreInstallInProgress, and SystemSetupInProgress values are set in that same HKLM\System\Setup registry key. In Windows PE, all three of those values are set to 1. In Windows PE, the CmdLine is prepopulated with this:



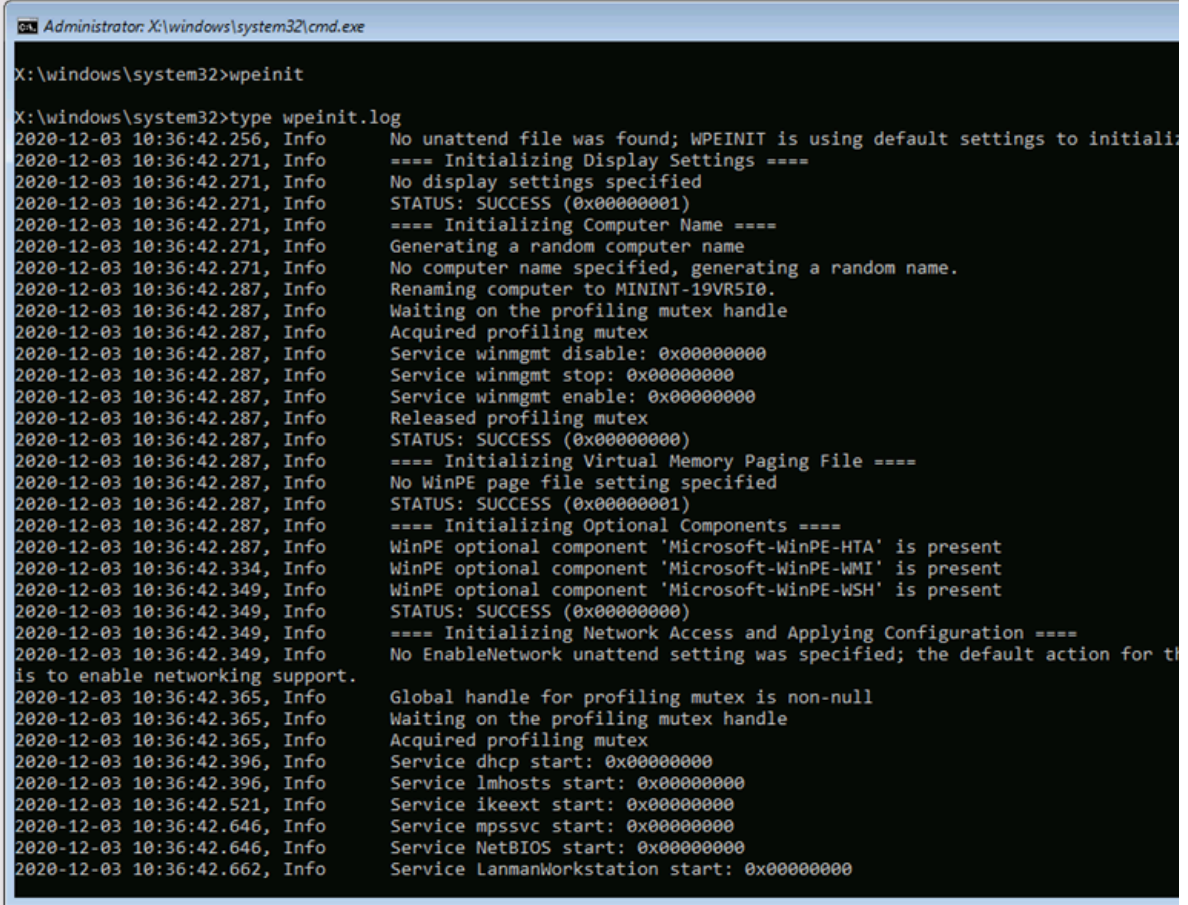
None of these registry values appear to be documented (no big surprise there). (As an aside, you might have seen these registry values before if you have used a ConfigMgr task sequence – it manipulates these values to make sure that SetupComplete.cmd runs after each reboot.)

Alright, so that causes winpeshl.exe to be executed. What does winpeshl.exe then do? It looks for a winpeshl.ini file that lists the commands that should be executed. If you look at the MDT installation files, you'll see that MDT uses a winpeshl.ini file like this:



Super-simple, just instructing winpeshl.exe to run bddrun.exe and pass it a “/bootstrap” parameter. But what if there is no winpeshl.ini file (which does not exist by default in Windows PE unless you add one)? In that case, winpeshl.exe will automatically run the command “%SYSTEMROOT%\System32\cmd.exe” with the parameter “/k startnet.cmd”. As you can probably guess, that will open a command prompt that runs whatever is in startnet.cmd.

So what is in startnet.cmd then? A single command, executing “wpeinit.exe”. By default, that will initialize networking, optional components, and a bunch of other stuff, so it’s essential to the operation of Windows PE. If you want to see the details of what it did, the X:\System32\wpeinit.log file is key:



```
Administrator: X:\windows\system32\cmd.exe

X:\windows\system32>wpeinit

X:\windows\system32>type wpeinit.log
2020-12-03 10:36:42.256, Info      No unattend file was found; WPEINIT is using default settings to initialize
2020-12-03 10:36:42.271, Info      ==== Initializing Display Settings ====
2020-12-03 10:36:42.271, Info      No display settings specified
2020-12-03 10:36:42.271, Info      STATUS: SUCCESS (0x00000001)
2020-12-03 10:36:42.271, Info      ==== Initializing Computer Name ====
2020-12-03 10:36:42.271, Info      Generating a random computer name
2020-12-03 10:36:42.271, Info      No computer name specified, generating a random name.
2020-12-03 10:36:42.287, Info      Renaming computer to MININT-19VR510.
2020-12-03 10:36:42.287, Info      Waiting on the profiling mutex handle
2020-12-03 10:36:42.287, Info      Acquired profiling mutex
2020-12-03 10:36:42.287, Info      Service winmgmt disable: 0x00000000
2020-12-03 10:36:42.287, Info      Service winmgmt stop: 0x00000000
2020-12-03 10:36:42.287, Info      Service winmgmt enable: 0x00000000
2020-12-03 10:36:42.287, Info      Released profiling mutex
2020-12-03 10:36:42.287, Info      STATUS: SUCCESS (0x00000000)
2020-12-03 10:36:42.287, Info      ==== Initializing Virtual Memory Paging File ====
2020-12-03 10:36:42.287, Info      No WinPE page file setting specified
2020-12-03 10:36:42.287, Info      STATUS: SUCCESS (0x00000001)
2020-12-03 10:36:42.287, Info      ==== Initializing Optional Components ====
2020-12-03 10:36:42.287, Info      WinPE optional component 'Microsoft-WinPE-HTA' is present
2020-12-03 10:36:42.334, Info      WinPE optional component 'Microsoft-WinPE-WMI' is present
2020-12-03 10:36:42.349, Info      WinPE optional component 'Microsoft-WinPE-WSH' is present
2020-12-03 10:36:42.349, Info      STATUS: SUCCESS (0x00000000)
2020-12-03 10:36:42.349, Info      ==== Initializing Network Access and Applying Configuration ====
2020-12-03 10:36:42.349, Info      No EnableNetwork unattend setting was specified; the default action for the
is to enable networking support.
2020-12-03 10:36:42.365, Info      Global handle for profiling mutex is non-null
2020-12-03 10:36:42.365, Info      Waiting on the profiling mutex handle
2020-12-03 10:36:42.365, Info      Acquired profiling mutex
2020-12-03 10:36:42.396, Info      Service dhcp start: 0x00000000
2020-12-03 10:36:42.396, Info      Service lmhosts start: 0x00000000
2020-12-03 10:36:42.521, Info      Service ikeext start: 0x00000000
2020-12-03 10:36:42.646, Info      Service mpssvc start: 0x00000000
2020-12-03 10:36:42.646, Info      Service NetBIOS start: 0x00000000
2020-12-03 10:36:42.662, Info      Service LanmanWorkstation start: 0x00000000
```

(Some of you might have looked at this file before because it can be useful to investigate why Windows PE is failing to initialize properly, e.g. when you added a bad drivers or didn’t allocate enough RAM. If one of the startup activities fails, WPEINIT will open a new command prompt to let you investigate.)

The first line of that log is particularly interesting: WPEINIT looks for an unattend.xml file to tell it what to do. Here’s an example from MDT:



```
Unattend_PE_x64.xml - Notepad
File Edit Format View Help
<?xml version="1.0" encoding="utf-8"?>
<unattend xmlns="urn:schemas-microsoft-com:unattend">
  <settings pass="windowsPE">
    <component name="Microsoft-Windows-Setup" processorArchitecture="amd64" publicKeyToken="31bf3856ad364e35" language="neutral" versionScope="nonSxS" xmlns:wcm="http://schemas.microsoft.com/W
    <Display>
      <ColorDepth>32</ColorDepth>
      <HorizontalResolution>1024</HorizontalResolution>
      <RefreshRate>60</RefreshRate>
      <VerticalResolution>768</VerticalResolution>
    </Display>
    <RunSynchronous>
      <RunSynchronousCommand wcm:action="add">
        <Description>Lite Touch PE</Description>
        <Order>1</Order>
        <Path>script.exe X:\Deploy\Scripts\LiteTouch.usfc\Path>
      </RunSynchronousCommand>
    </RunSynchronous>
  </component>
</settings>
</unattend>
```

If you want to see all of the available settings, use the Windows System Image Manager to see what is available in the “windowsPE” settings pass. Only those settings will be processed. In the MDT case, the file just sets the screen resolution (although I’m not 100% sure that works all the time) and runs “wscript.exe X:\Deploy\Scripts\LiteTouch.wsf” to start or resume the MDT process.

So let’s review the default process:

- winlogon.exe runs winpeshl.exe.
- winpeshl.exe runs “cmd.exe /k startnet.cmd”
- startnet.cmd runs wpeinit.exe.
- You’re left sitting at a command prompt, and Windows PE will reboot when that command prompt exits (as long as it sets an exit code of 0).

And you can modify every step of that process:

- You could change HKLM\System\Setup\CmdLine to run something other than winpeshl.exe, but that’s not recommended/supported.
- You could create your own winpeshl.ini to specify commands to run instead of “cmd.exe /k startnet.cmd”. That’s highly recommended (more on that later).
- You could modify “startnet.cmd” to run your own stuff. That’s quick-and-dirty, but works fine (more on that later).
- You could add an unattend.xml that gets processed by wpeinit.exe. That’s highly recommended too, as you probably don’t want to run anything until networking and optional components are initialized.

Now look at how MDT does it:

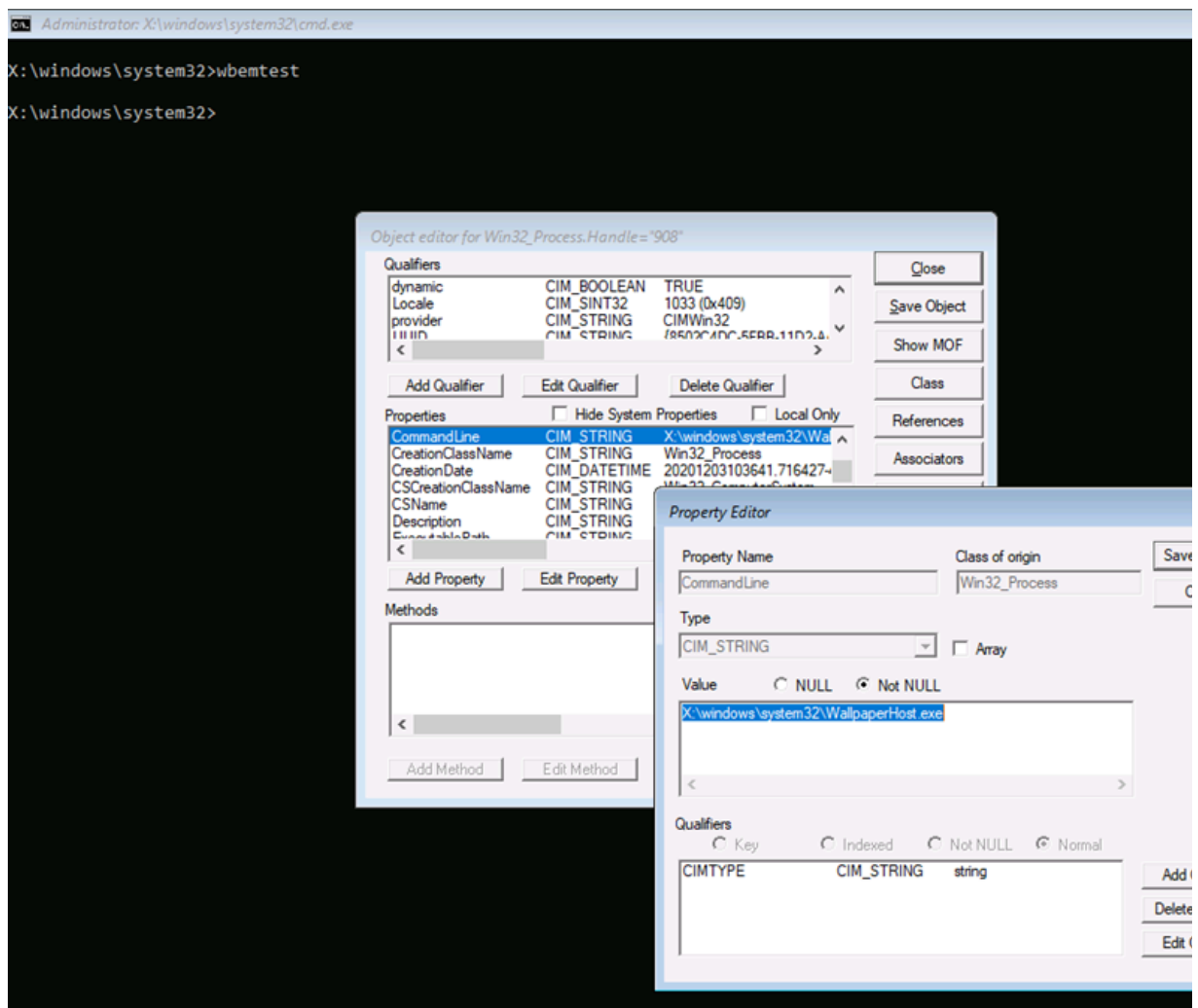
- winlogon.exe runs winpeshl.exe (default).
- winpeshl.exe finds winpeshl.ini, and runs the “bddrun.exe /bootstrap” command in it.
- bddrun.exe runs wpeinit.exe itself (more on that later).
- wpeinit.exe finds an unattend.xml, and runs the “wscript.exe X:\Deploy\Scripts\LiteTouch.wsf” command in it.
- When wscript.exe exits, the machine reboots.

It’s worth explaining why that’s being done:

- If you didn’t have a winpeshl.ini, it would run “cmd.exe /k startnet.cmd” and that would cause a visible command prompt to appear on the screen. If you specify a Windows (non-console) app in winpeshl.ini, it can run “invisibly.”
- wpeinit.exe is essential, hence bddrun.exe executes it (invisibly).
- wpeinit.exe runs wscript.exe (not cscript.exe) so that the script host runs invisibly too (if it wants to display something it runs an HTA or a popup – no console access).

So this is all to create a nice, clean “visual effect.”

While we’re going through all of this, there’s one other item worth mentioning: The background wallpaper. By default, Windows PE will use the “%SystemRoot%\System32\winpe.bmp” for the background, and it finds that path from the registry at HKLM\Software\Microsoft\Windows NT\CurrentVersion\WinPE\CustomWallpaper. So if you changed that registry key in your Windows PE boot image, it would use that instead. Or you can cheat and just overlay the winpe.bmp with your own content and it will display that instead (which is what MDT does). If we were in the full Windows OS, it would be the shell (Explorer.exe) that displayed the wallpaper, but that’s not running in Windows PE. So what displays the wallpaper? That’s one of the other things that wpeinit.exe kicks off: It starts a WallpaperHost.exe process that takes the place of Explorer. You can see it running in Windows PE:



That then enables wpeinit.exe to call a standard API to set the wallpaper to the file specified.

Subscribe to get the latest posts sent to your email.

SUBSCRIBE

AUTHOR



Written by

Michael Niehaus

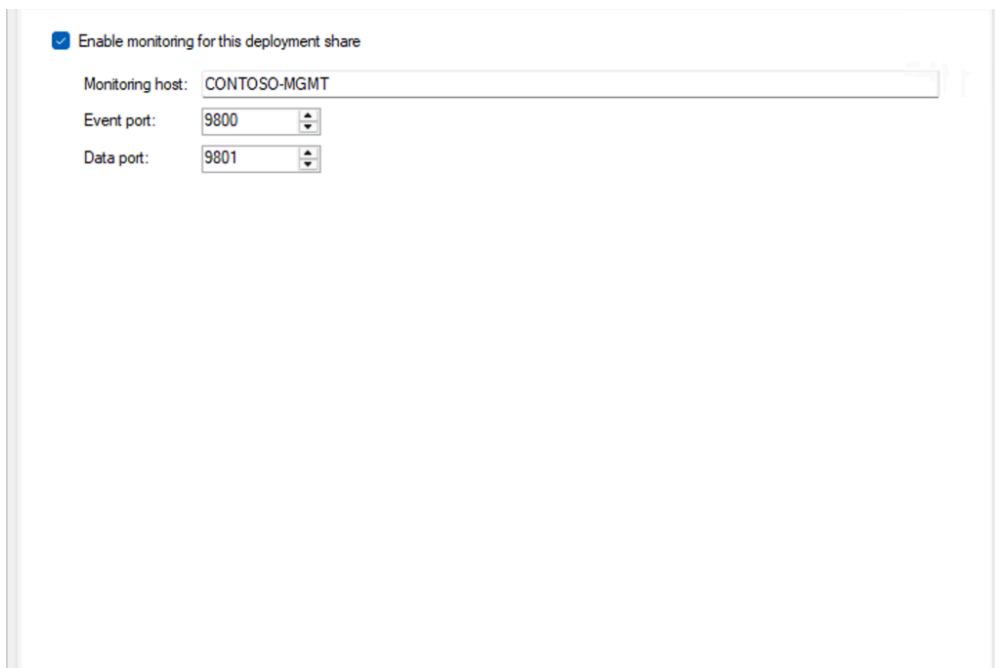
Tech geek, deployment guy, provisioning expert, device management practitioner, product manager, developer

RECENT POSTS

DeployR

Looking for an MDT replacement? Consider DeployR.

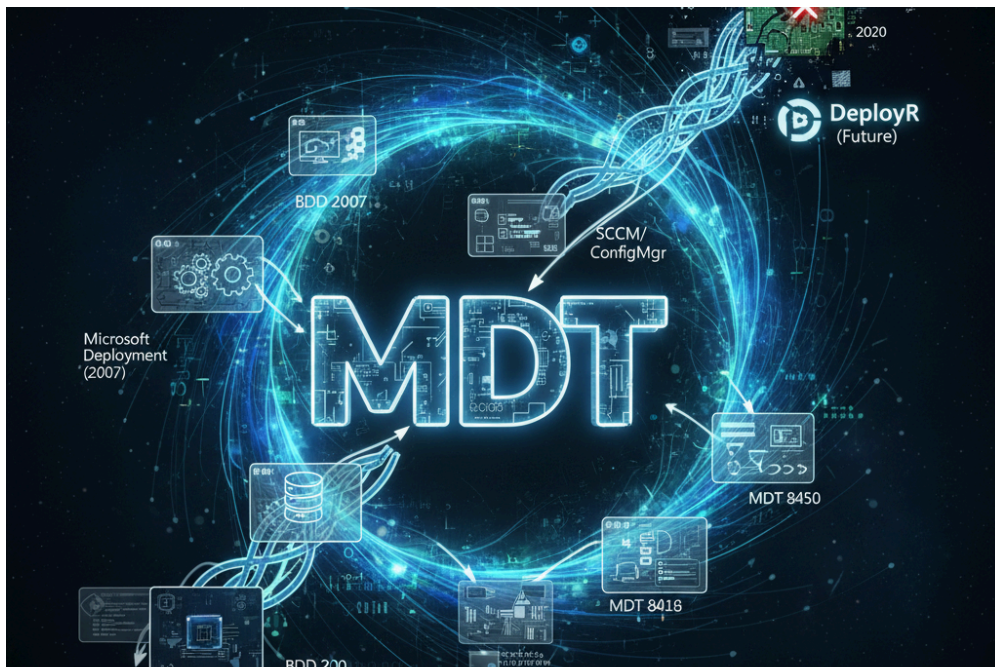
Michael Niehaus



Microsoft Deployment Toolkit

Following up on the MDT security issue

Michael Niehaus



Microsoft Deployment Toolkit

A complete history of MDT (abridged)

Michael Niehaus

CATEGORIES

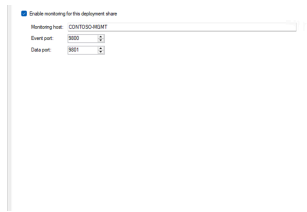
- [Azure](#)
- [Azure Active Directory](#)
- [DeployR](#)
- [Geeking Out](#)
- [Linux](#)
- [MacOS](#)
- [Microsoft 365](#)
- [Microsoft Configuration Manager](#)
- [Microsoft Deployment Toolkit](#)
- [Microsoft Graph](#)
- [Microsoft Intune](#)
- [Miscellaneous](#)
- [Office 365 ProPlus](#)
- [OneDrive for Business](#)
- [PC Hardware](#)
- [PowerShell](#)
- [System Center Configuration Manager](#)
- [Tanium](#)
- [Uncategorized](#)
- [Windows](#)
- [Windows 10](#)
- [Windows 11](#)
- [Windows 365](#)
- [Windows as a Service](#)
- [Windows Autopilot](#)
- [Windows Server](#)

TRENDING

DeployR

Looking for an MDT replacement? Consider DeployR.

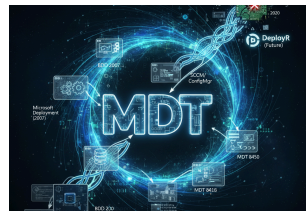
Michael Niehaus



Microsoft Deployment Toolkit

Following up on the MDT security issue

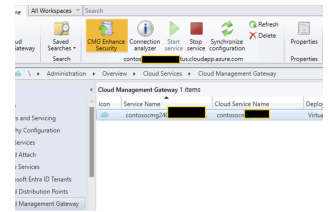
Michael Niehaus



Microsoft Deployment Toolkit

A complete history of MDT (abridged)

Michael Niehaus



Microsoft Configuration Manager

Still running ConfigMgr tech preview? Not for long...

Michael Niehaus

[Home](#)

[RSS](#)

[SUBSCRIBE](#)

[Log in](#)